

Linux Class 11 - Task

Deleting with sed, http and https, hostname, and Class Notes

Submitted by: Shivakumar

Date: 03 October 2026

For this task I made a folder called class11. Inside it I typed a file called data.txt in vi, with eight lines, each one the name of a dish. The word masala comes on line 1, line 4, line 6 and line 7, which is useful because then I can see clearly which lines the command worked on. I also made a second file called blanks.txt with some empty lines in it, for the second question.

A closed file means a file I have not opened in vi. sed does the whole job from the command line, so the file is never opened.

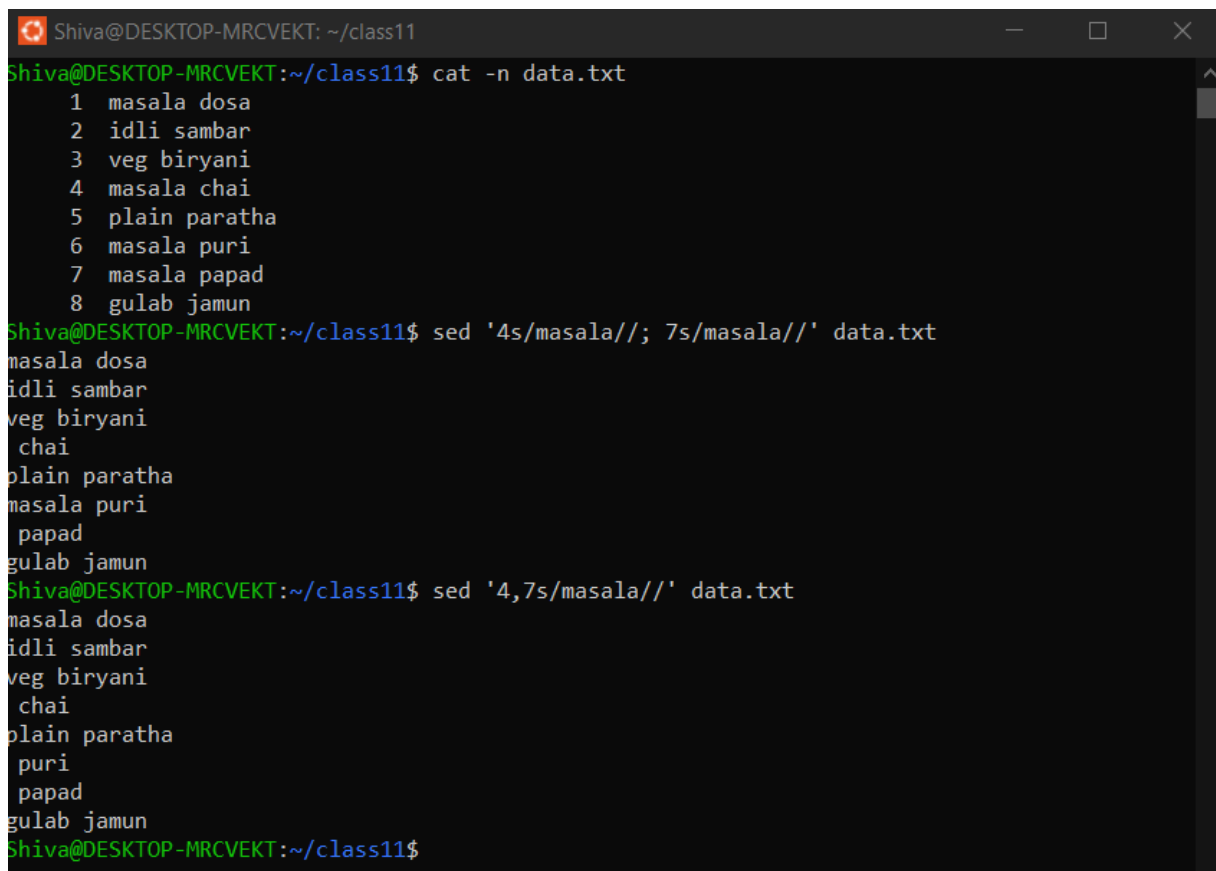
1. How to delete the pattern only in the 4th line and the 7th line of a closed file?

The command is:

```
sed '4s/masala//; 7s/masala//' data.txt
```

To delete a word, I use the same s command that replaces a word, but I leave the new word empty. In s/masala// there is nothing between the last two slashes, so masala is replaced by nothing, which means it is deleted.

The number in front of the s is the line number. So 4s works only on line 4 and 7s works only on line 7. The semicolon in the middle is what lets me give both instructions in one command.



```
Shiva@DESKTOP-MRCVEKT: ~/class11
Shiva@DESKTOP-MRCVEKT:~/class11$ cat -n data.txt
 1 masala dosa
 2 idli sambar
 3 veg biryani
 4 masala chai
 5 plain paratha
 6 masala puri
 7 masala papad
 8 gulab jamun
Shiva@DESKTOP-MRCVEKT:~/class11$ sed '4s/masala//; 7s/masala//' data.txt
masala dosa
idli sambar
veg biryani
chai
plain paratha
masala puri
papad
gulab jamun
Shiva@DESKTOP-MRCVEKT:~/class11$ sed '4,7s/masala//' data.txt
masala dosa
idli sambar
veg biryani
chai
plain paratha
puri
papad
gulab jamun
Shiva@DESKTOP-MRCVEKT:~/class11$
```

Screenshot 1: masala is gone from lines 4 and 7, but line 1 and line 6 still have it.

In the answer, line 4 became just chai and line 7 became just papad, but line 1 is still masala dosa and line 6 is still masala puri. That proves the command touched only the two lines I asked for.

The second command in the screenshot shows the mistake I made first. I wrote a comma instead of a semicolon:

```
sed '4,7s/masala//' data.txt
```

A comma means a range, so that one works on line 4, line 5, line 6 and line 7 together. In its answer line 6 also lost the word and became just puri, which is wrong for this question. So the semicolon is the important part here.

Both of these only show the answer on the screen. To save it into the file I add -i:

```
sed -i '4s/masala//; 7s/masala//' data.txt
```

2. How to remove blank lines using the sed command?

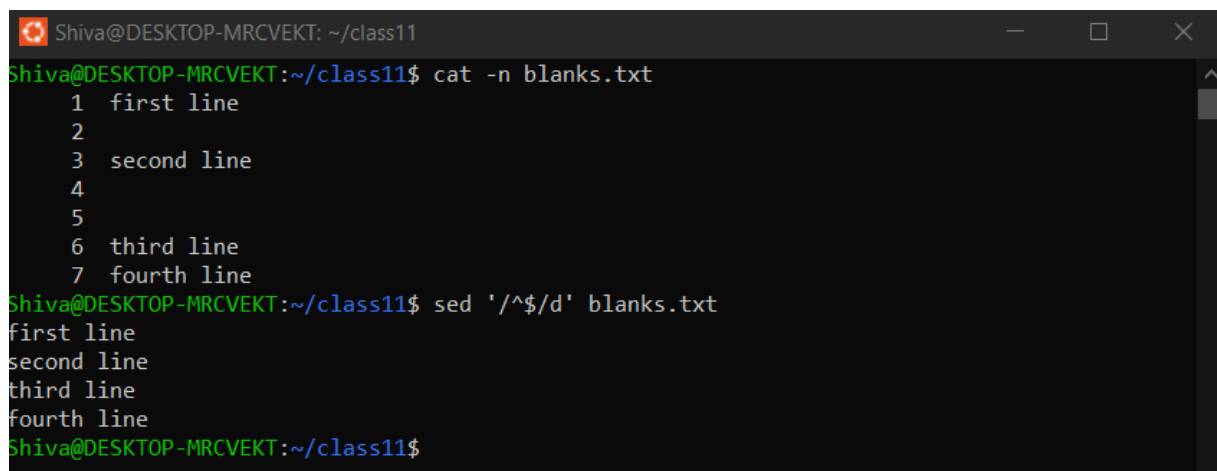
The command is:

```
sed '/^$/d' blanks.txt
```

The part inside the slashes is what sed looks for, and d at the end means delete the line. So I have to describe a blank line to sed:

- ^ means the start of the line.
- \$ means the end of the line.
- Writing them together with nothing in between means a line where the start is also the end, which is a blank line.

My file had 7 lines with blank lines at 2, 4 and 5. After the command only the four real lines were left.

A screenshot of a terminal window with a dark background. The window title is 'Shiva@DESKTOP-MRCVEKT: ~/class11'. The first command is 'cat -n blanks.txt', which outputs a file with 7 lines: line 1 is 'first line', line 2 is blank, line 3 is 'second line', line 4 is blank, line 5 is blank, line 6 is 'third line', and line 7 is 'fourth line'. The second command is 'sed '/^\$/d' blanks.txt', which outputs the file with blank lines removed: 'first line', 'second line', 'third line', and 'fourth line'. The prompt 'Shiva@DESKTOP-MRCVEKT:~/class11\$' is visible at the bottom.

Screenshot 2: the file with its blank lines, and then the same file with them removed.

Again, to save it into the file I add -i:

```
sed -i '/^$/d' blanks.txt
```

3. How to delete the lines where the pattern is present using sed?

The command is:

```
sed '/masala/d' data.txt
```

This is the same shape as the blank line command. Inside the slashes I put the word to look for, and d deletes the whole line. My file had masala on four lines, so those four lines disappeared and only the other four were left.

```
Shiva@DESKTOP-MRCVEKT: ~/class11
Shiva@DESKTOP-MRCVEKT:~/class11$ sed '/masala/d' data.txt
idli sambar
veg biryani
plain paratha
gulab jamun
Shiva@DESKTOP-MRCVEKT:~/class11$ cp data.txt copy.txt
Shiva@DESKTOP-MRCVEKT:~/class11$ sed -i '/masala/d' copy.txt
Shiva@DESKTOP-MRCVEKT:~/class11$ cat copy.txt
idli sambar
veg biryani
plain paratha
gulab jamun
Shiva@DESKTOP-MRCVEKT:~/class11$
```

Screenshot 3: the four masala lines are gone, and the -i version saves it into the file.

In the screenshot I also made a copy first and then used -i on the copy, so that my original data.txt was not spoiled:

```
cp data.txt copy.txt
sed -i '/masala/d' copy.txt
```

I have to be careful about between question 1 and question 3. They look similar but they do different things:

Command	What it removes
sed '4s/masala//' data.txt	Only the word masala on line 4. The rest of the line stays.
sed '/masala/d' data.txt	The whole line, every time masala is found in it.
sed '/^\$/d' blanks.txt	Every blank line.

So s/word// takes the word out and keeps the line, and /word/d throws the whole line away.

4. What is the difference between http and https?

Both of them are the rules a browser uses to ask a website for a page. The difference is only one letter, the s, and that s stands for secure.

With http the information travels as plain text. Anybody sitting in between, for example on the same wifi, can read it. With https the information is encrypted first, so even if somebody catches it, they cannot understand it.

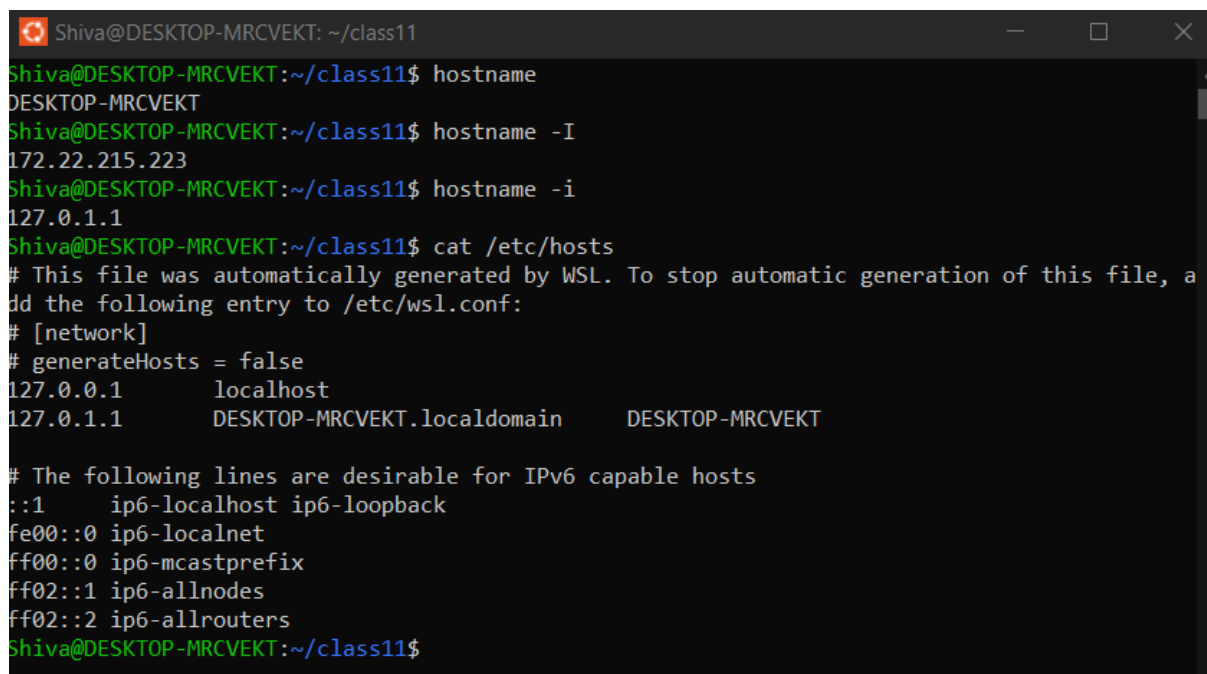
	http	https
Full name	HyperText Transfer Protocol	The same, with Secure at the end
Port number	80	443
Is it safe	No. The data goes as plain text.	Yes. The data is encrypted.
Certificate	Not needed.	Needs an SSL certificate.
In the browser	Shows Not secure.	Shows a padlock.
Used for	Ordinary pages with nothing private.	Logins, payments, anything private.

That is why every site where I type a password or a card number must be https. The browser shows a small padlock next to the address when it is https, and says Not secure when it is only http.

5. What is the difference between hostname -I and hostname -i?

hostname on its own just prints the name of the machine. The two options then give two different IP addresses, and they are easy to mix up because they are the same letter in capital and small.

Command	What it shows
hostname	The name of the machine. Mine said DESKTOP-MRCVEKT.
hostname -I	The real IP address on the network. Capital I. Mine said 172.22.215.223.
hostname -i	The IP that the name points to in /etc/hosts. Small i. Mine said 127.0.1.1.



```
Shiva@DESKTOP-MRCVEKT: ~/class11
Shiva@DESKTOP-MRCVEKT:~/class11$ hostname
DESKTOP-MRCVEKT
Shiva@DESKTOP-MRCVEKT:~/class11$ hostname -I
172.22.215.223
Shiva@DESKTOP-MRCVEKT:~/class11$ hostname -i
127.0.1.1
Shiva@DESKTOP-MRCVEKT:~/class11$ cat /etc/hosts
# This file was automatically generated by WSL. To stop automatic generation of this file, add
the following entry to /etc/wsl.conf:
# [network]
# generateHosts = false
127.0.0.1    localhost
127.0.1.1    DESKTOP-MRCVEKT.localdomain    DESKTOP-MRCVEKT

# The following lines are desirable for IPv6 capable hosts
::1        ip6-localhost ip6-loopback
fe00::0    ip6-localnet
ff00::0    ip6-mcastprefix
ff02::1    ip6-allnodes
ff02::2    ip6-allrouters
Shiva@DESKTOP-MRCVEKT:~/class11$
```

Screenshot 4: the two options give different answers, and /etc/hosts shows why.

In my screenshot capital I gave 172.22.215.223, which is the real address my machine has on the network. Small i gave 127.0.1.1, which is only a loopback address, meaning this machine itself.

The last command explains where the small i answer came from. I opened the hosts file:

```
cat /etc/hosts
```

Inside it there is a line 127.0.1.1 DESKTOP-MRCVEKT. hostname -i simply looks up the machine name in that file and prints whatever address is written next to it. hostname -I does not use that file at all, it asks the network cards directly.

So capital I is the one to use when I want to know the real address of the machine. Small i can give a useless answer like 127.0.1.1.

Other commands covered in this class

ps and kill

ps shows the processes that are running. It is a snapshot, which means it shows what was running at the moment I pressed Enter, and it does not keep updating. The top command is the one that keeps updating.

- ps on its own shows only the processes of my own terminal.
- ps -ef shows all the processes of all the users on the system.
- In -ef, the e means every process and the f means full details.

To try it I started a process that does nothing for 300 seconds, and put & at the end so that it runs in the background and gives my prompt back:

```
sleep 300 &
```

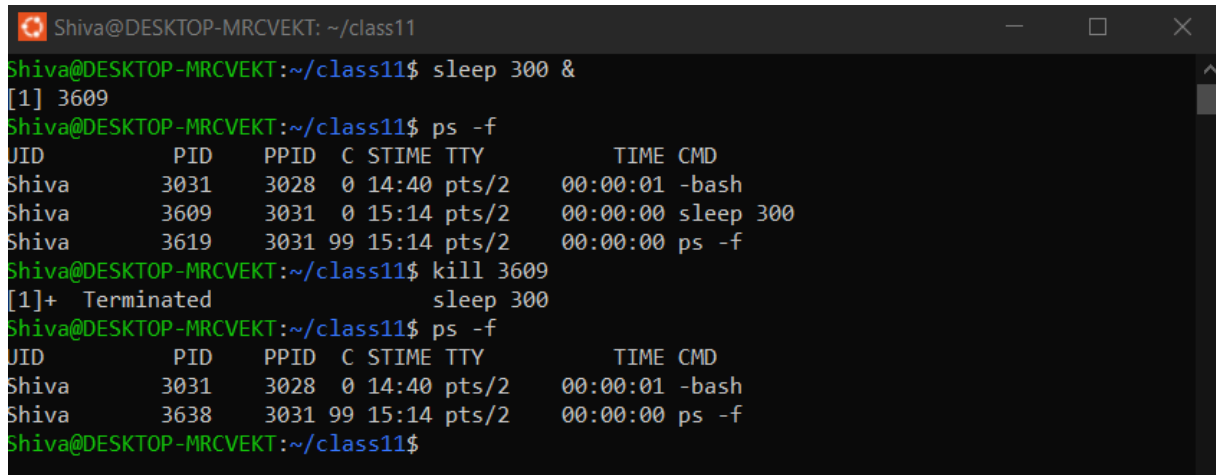
```
ps -f
```

ps -f showed three things: my bash, the sleep, and the ps command itself. The number in the PID column is the process id, which is what I need to stop it. For my sleep it was 3609:

```
kill 3609
```

It printed Terminated, and when I ran ps -f again the sleep was gone. If a process does not stop with plain kill, there is a stronger form:

```
kill -9 3609
```

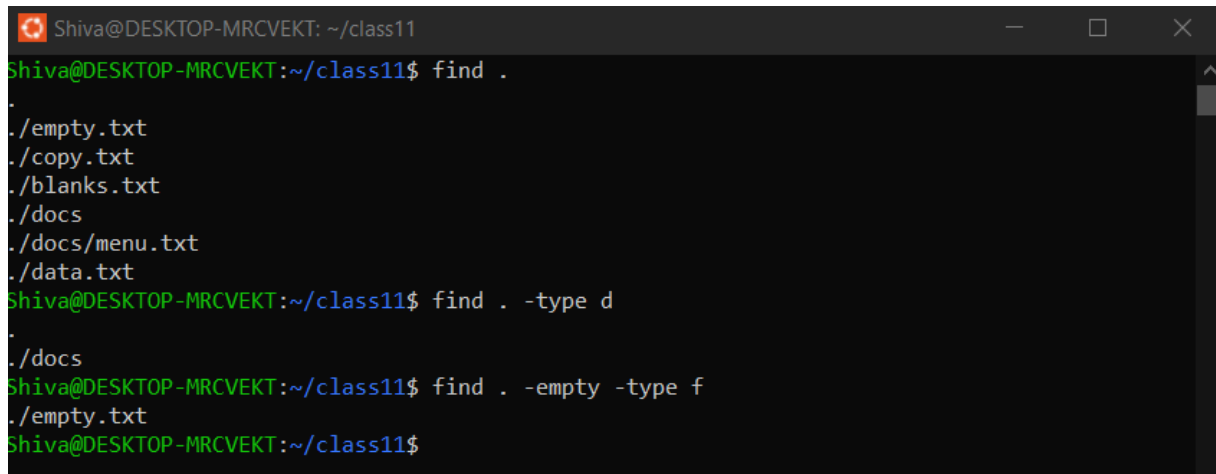
A terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/class11'. The user runs 'sleep 300 &', which returns '[1] 3609'. Then they run 'ps -f', showing a list of processes including 'sleep 300' with PID 3609. Next, they run 'kill 3609', which returns '[1]+ Terminated sleep 300'. Finally, they run 'ps -f' again, and the 'sleep 300' process is no longer listed.

```
Shiva@DESKTOP-MRCVEKT: ~/class11
Shiva@DESKTOP-MRCVEKT:~/class11$ sleep 300 &
[1] 3609
Shiva@DESKTOP-MRCVEKT:~/class11$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
Shiva        3031     3028  0 14:40 pts/2        00:00:01 -bash
Shiva        3609     3031  0 15:14 pts/2        00:00:00 sleep 300
Shiva        3619     3031  99 15:14 pts/2        00:00:00 ps -f
Shiva@DESKTOP-MRCVEKT:~/class11$ kill 3609
[1]+  Terminated                  sleep 300
Shiva@DESKTOP-MRCVEKT:~/class11$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
Shiva        3031     3028  0 14:40 pts/2        00:00:01 -bash
Shiva        3638     3031  99 15:14 pts/2        00:00:00 ps -f
Shiva@DESKTOP-MRCVEKT:~/class11$
```

Screenshot 5: a background process started, found with ps -f, and then killed.

find

find lists what is inside a folder, including everything in the folders inside it. On its own, find . shows files, folders and links all together, and then the options narrow it down.

A terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/class11'. The user runs 'find .' which lists files like 'empty.txt', 'copy.txt', 'blanks.txt', 'docs', 'docs/menu.txt', and 'data.txt'. Then they run 'find . -type d', which lists only the directory 'docs'. Finally, they run 'find . -empty -type f', which lists only the empty file 'empty.txt'.

```
Shiva@DESKTOP-MRCVEKT: ~/class11
Shiva@DESKTOP-MRCVEKT:~/class11$ find .
.
./empty.txt
./copy.txt
./blanks.txt
./docs
./docs/menu.txt
./data.txt
Shiva@DESKTOP-MRCVEKT:~/class11$ find . -type d
.
./docs
Shiva@DESKTOP-MRCVEKT:~/class11$ find . -empty -type f
./empty.txt
Shiva@DESKTOP-MRCVEKT:~/class11$
```

Screenshot 6: everything, then only the folders, then only the empty files.

Command	What it shows
find .	Everything in the folder I am in, including what is inside sub folders.
find /home/Shiva	The same, but starting from the path I give it.
find . -type f	Only the files.
find . -type d	Only the folders.
find . -type l	Only the links.

Command	What it shows
<code>find . -empty -type f</code>	Only the empty files.
<code>find . -empty -type d</code>	Only the empty folders.
<code>find . -perm 777</code>	Everything that has 777 permission.
<code>find . -mmin -15</code>	Everything changed in the last 15 minutes.
<code>find . -mmin +15</code>	Everything changed more than 15 minutes ago.

A small note on the time options. -mmin is when the file was last modified, -amin is when it was last accessed, and -cmin is when the details of the file last changed. A plus sign means older than that many minutes and a minus sign means within that many minutes, so -mmin -15 is the last 15 minutes and -mmin +15 is before that.

The options can also be joined together, for example `find . -empty -type f` means empty and a file, not a folder.

Commands I used

Command	What it does
<code>sed '4s/word//; 7s/word//' file</code>	Deletes the word on line 4 and line 7 only.
<code>sed '/^\$/d' file</code>	Deletes all the blank lines.
<code>sed '/word/d' file</code>	Deletes every line that has the word in it.
<code>sed -i</code>	Saves the change into the file instead of only showing it.
<code>hostname</code>	The name of the machine.
<code>hostname -I</code>	The IP address on the network.
<code>hostname -i</code>	The IP from the /etc/hosts file.
<code>ps</code>	The processes running in this terminal.
<code>ps -ef</code>	All the processes of all the users.
<code>kill 3609</code>	Stops the process with that id.
<code>kill -9 3609</code>	Stops it forcefully, when kill alone does not work.
<code>find . -type f</code>	Lists only the files.